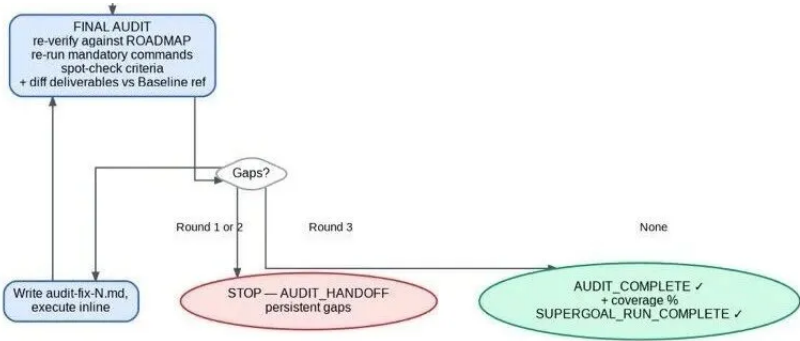




Member-only story

Introducing /supergoal: The Powerful New Plugin for Claude

 AI Engineering

Follow

 5 min read · Jun 6, 2026

👏 210

💬 2

↺ 1

🔖

▶

📄

⋮

By winkrun, AI Engineering | June 6, 2026

Developer Robert Courson just launched /supergoal, a game-changing new plugin for Claude Code and Codex CLI that solves one of the most

frustrating pain points of AI-powered programming assistants.

Unlike small utility tools that just handle single tasks like “write me a function,” /supergoal targets a much more specific problem: the constant back-and-forth approval process that every AI coding assistant forces you to go through.

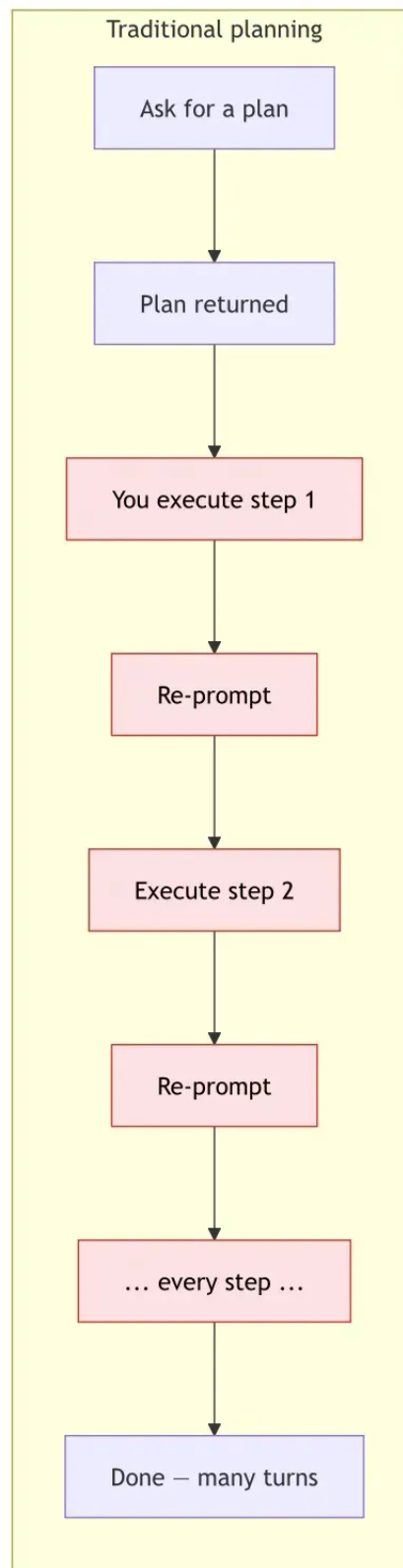
If you’re wondering why we need another plugin when Claude Code already has the built-in /goal command, you're not alone. But the difference between the two boils down to one simple thing:

The only real difference is who handles the project planning. Execution autonomy works exactly the same for both — both rely on the same /goal execution loop to get the job done.

Let me break this down into three core layers to make it clear:

1. /goal is an execution engine — it needs you to give it a clear end condition

When you use the native /goal command, you feed it a single end-state condition (like "get all tests in test/auth passing and clean up all lint errors") and it runs autonomously until that condition is met. The problem? You're on the hook for figuring out the condition, breaking the project into logical steps, and ordering the work correctly before you ever run the command. If your condition is vague or your plan is messy, the AI will go off the rails fast.



It's not uncommon to hear developers complain that they rewrote their

`/goal` condition three times just to get a single project to run correctly. That's the exact pain point `/supergoal` solves.

2. `/supergoal` is a planning engine — it does all the upfront work for you

`/supergoal` handles all the pre-planning work that `/goal` doesn't touch: it scans your codebase, identifies potential risks, pulls from past project memory, breaks the full task into multiple phases, writes each phase's detailed specification to a `.supergoal` directory on your disk, does a self-audit to catch gaps, and finally spits out a polished, ready-to-use `/goal` command that you just copy and paste.

That 7-hour autonomous run you might have heard about? The autonomy comes from `/goal` — that's not an invention from `/supergoal`. `/supergoal` just sets it up to succeed.

3. `/supergoal` adds an extra layer of self-healing and auditing

This is something the native `/goal` doesn't have out of the box:

- A three-strike error tolerance system: detect failure and retry → write a targeted fix specification → pause and hand back control to you if it still fails
- Automatic memory logging after every phase
- A full final audit against the original roadmap to diff-check that every deliverable actually got built

All of this extra logic is stored as plain files on your disk, which the `/goal` loop reads and executes automatically.

The full /supergoal workflow in action

All you need to do is input one simple command, like this:

```
/supergoal build me an Expo app that converts photos to ASCII art
```

From there, /supergoal handles every step of the planning process automatically:

1. **Stage 0** — Load past project memory and detect all tools available in your current environment (MCP servers, skills, host configuration)
2. **Stage 1** — Detect if you're working on a brand-new greenfield project or modifying an existing brownfield codebase. For new projects, it walks through a full checklist (platform, tech stack, design requirements, integrations, scope...). For existing projects, it only asks 0–2 clarification questions because the codebase itself already answers most questions
3. **Stage 2** — Scan your entire codebase in parallel to map the project structure
4. **Stage 3** — Analyze potential risks and recommend best practices for your

specific project

5. **Stage 4** — Break the full task into an adaptive number of phases (no fixed step count, it adjusts to your project size)
6. **Stage 5** — Write the full roadmap and phase-specific specifications to the `.supergoal/` directory on your disk
7. **Stage 6** — Run a self-criticism check to catch vague goals and bad task splitting, fixes issues on the spot, then shows you the reviewed plan with an edit menu (start, adjust assumptions, rework phases, restructure steps)
8. **Stage 6.5** — Run a pre-flight check: executes all required baseline commands (build, lint, test) to confirm your starting environment is clean
9. **Stage 7** — After everything passes, it prints a ready-to-paste `/goal <end-state condition>` command. You just copy, paste once, and the rest runs fully automatically

During the automatic execution phase, a 3-strike fault tolerance system kicks in:

- First failure: automatically retry with extra debugging context injected
- Second failure: generate a focused fix specification and execute it
- Third failure: pause, and hand you the full debugging history to figure out what went wrong

After every phase, /supergoal writes project knowledge to memory: it records non-obvious details like tricky API gotchas, your personal preferences, and project-specific facts that the AI can pull from for future similar tasks.

When everything finishes running, it runs one final audit: it cross-checks all deliverables against the original roadmap, re-runs all required baseline checks, and confirms every deliverable is actually implemented (it even checks uncommitted changes). Only after everything passes does it print `SUPERGOAL_RUN_COMPLETE` .

One-sentence summary

/goal gives you "autonomous execution", /supergoal gives you "figure out the plan upfront + self-fix when things go wrong + self-audit at the end".

/goal is the low-level primitive, /supergoal is the planning and quality-check scaffolding that wraps around it. For small bug fixes and minor changes, you can just stick with plain /goal . For large new greenfield projects or big refactors, /supergoal eliminates all the mental work you'd otherwise spend writing end conditions and breaking work into phases.

Installation

For Claude Code, just run these three commands:

```
/plugin marketplace add  
https://github.com/robzilla1738/supergoal.git  
/plugin install supergoal@supergoal  
/reload-plugins
```

For Codex CLI, you'll need to manually clone the repository into your skills directory.

You can find the full GitHub repository here:

<https://github.com/robzilla1738/supergoal>

Ai Programming

Claude Code

Developer Tools

Ai Engineering

Productivity



Written by AI Engineering

635 followers · 33 following

Follow

Sharing of cutting-edge AI product technology information and experience!

<https://apps.apple.com/us/app/wink-pings/id6751033893> download now!

